

Architectural Design

Architectural Design: Architectural Design
Decisions, System organization Modular
Decomposition styles, Control styles
Object oriented design: Objects and Object
Classes, An object-oriented design process
Design evolution.

Introduction

- In software engineering, architectural design refers to the high-level structuring of a software system.
- It outlines how the system components interact with each other and how they are organized.
- The goal is to ensure that the software is scalable, maintainable, efficient and meets user requirements.

Introduction

- Large systems are generally divided into smaller sub-systems that provide a related set of services.
- This process of identifying the sub-systems and establishing a framework of communication between them is called **architectural design**.
- The architectural design process is concerned with **establishing a basic structural framework** that identifies the major components of a system and the communications between these components.

- Advantages of developing a system architecture:
 - Stakeholder communication
 - System analysis
 - Large scale reuse

- Architectural design decisions affect the following non-functional requirements:
 - Performance
 - Security
 - Availability
 - Maintainability
 - Safety

Architectural Design Decisions

- During the architectural design process, system architects have to make a number of fundamental decisions that profoundly affect the system and its development process:
 - Is there a generic application architecture that can act as a template for the system that is being designed?
 - How will the system be distributed across a number of processors?
 - What architectural style or styles are appropriate for the system?
 - What will be the fundamental approach used to structure the system?
 - How will the structural units in the system be decomposed into modules?
 - What strategy will be used to control the operation of the units in the system?
 - How will the architectural design be evaluated?
 - How should the architecture of the system be documented?

Contd..

- The architecture of the system depends on a particular architectural style. Architectural style refers to the way in which different system components are organized.
- For example: Client-server architecture, layered architecture.
- Depending on the system requirements, an appropriate architecture needs to be chosen.

Contd..

- The output of architectural design phase is a architectural **design document** containing a number of graphical representations that describe the different system components.
 - A **static structural model** that shows the sub-systems or components that are to be developed as separate units.
 - Example In an **online learning platform**, the structural model may include:
 - Authentication module
 - Course management system
 - Payment service
 - User dashboardEach component is developed independently and connected via well-defined APIs.
 - A **dynamic process model** that shows how the system is organized into processes at run-time. This may be different from the static model.
 - In a **multiplayer game server**, dynamic processes might include:
 - Game state synchronizer
 - Chat handler
 - Matchmaker process
 - Real-time physics engineEach runs in parallel or asynchronously depending on real-time conditions.

- An **interface model** that defines the services offered by each sub-system through its public interface.
- **Relationship models** that shows relationships, such as data flow, between the sub-systems.
- A **distribution model** that shows how sub-systems may be distributed across computers.

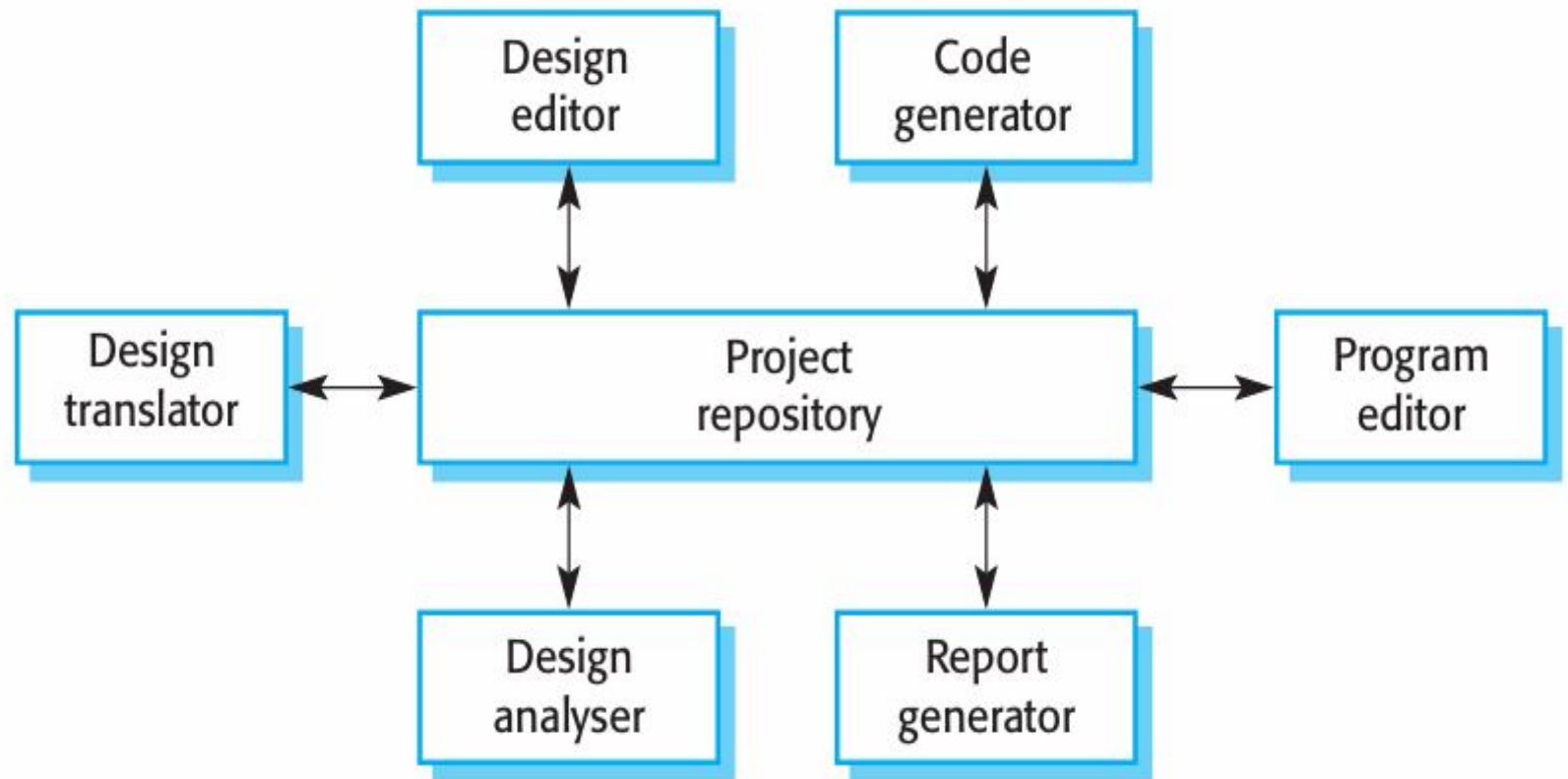
Repository Model

- The **Repository Model** is one of the **architectural styles** used in software engineering, especially when dealing with systems that need to **share and manage large volumes of data** across different components.

There are two fundamental ways to do this:

- **Shared repository model:** The data shared by all sub-systems is stored in a central database, called a shared repository.
- **Message passing model:** Every sub-system stores its own data and sub-systems communicate using message passing.

Contd..



Example

Compiler Design

A **compiler** is a classic example that often follows the **repository model**.

How it works:

A compiler has multiple **phases or components**, such as:

Lexical Analyzer (Scanner) Syntax Analyzer (Parser) Semantic Analyzer, Intermediate Code Generator, Code Optimizer, Code Generator.

All of these components operate on or interact with shared structures like:

Symbol Table

Abstract Syntax Tree (AST)

Intermediate Representations (IR)

Repository:

The **Symbol Table** and **AST** serve as the **central repository** that all components read from or write to.

Why it fits the Repository Model:

Each component is **independent** and focuses on its specific task.

They **do not communicate directly** with each other.

They access **shared data structures** (symbol table, AST) for input/output.

Data is **centrally managed** in one place.

Advantages

Centralized data management

Consistency of shared data

Easier integration of new tools/components

Promotes **reusability**

Disadvantages

Bottleneck at the repository if many tools access it heavily

Single point of failure (if the repository goes down, everything fails)

Possible **scalability issues**

Can lead to **tight data coupling** — changes to data format can break many components

- **Advantages and disadvantages of repository model:**

- It is an efficient way to share large amounts of data. There is no need to transmit data explicitly from one sub-system to another. However, all sub-systems must agree upon a common data model. This can be a compromise on the specific needs.
- Sub-systems that produce data need not be concerned with how that data is used by other sub-systems. But, when large amounts of data is generated by a sub-system and the data model changes, it would be difficult to migrate.

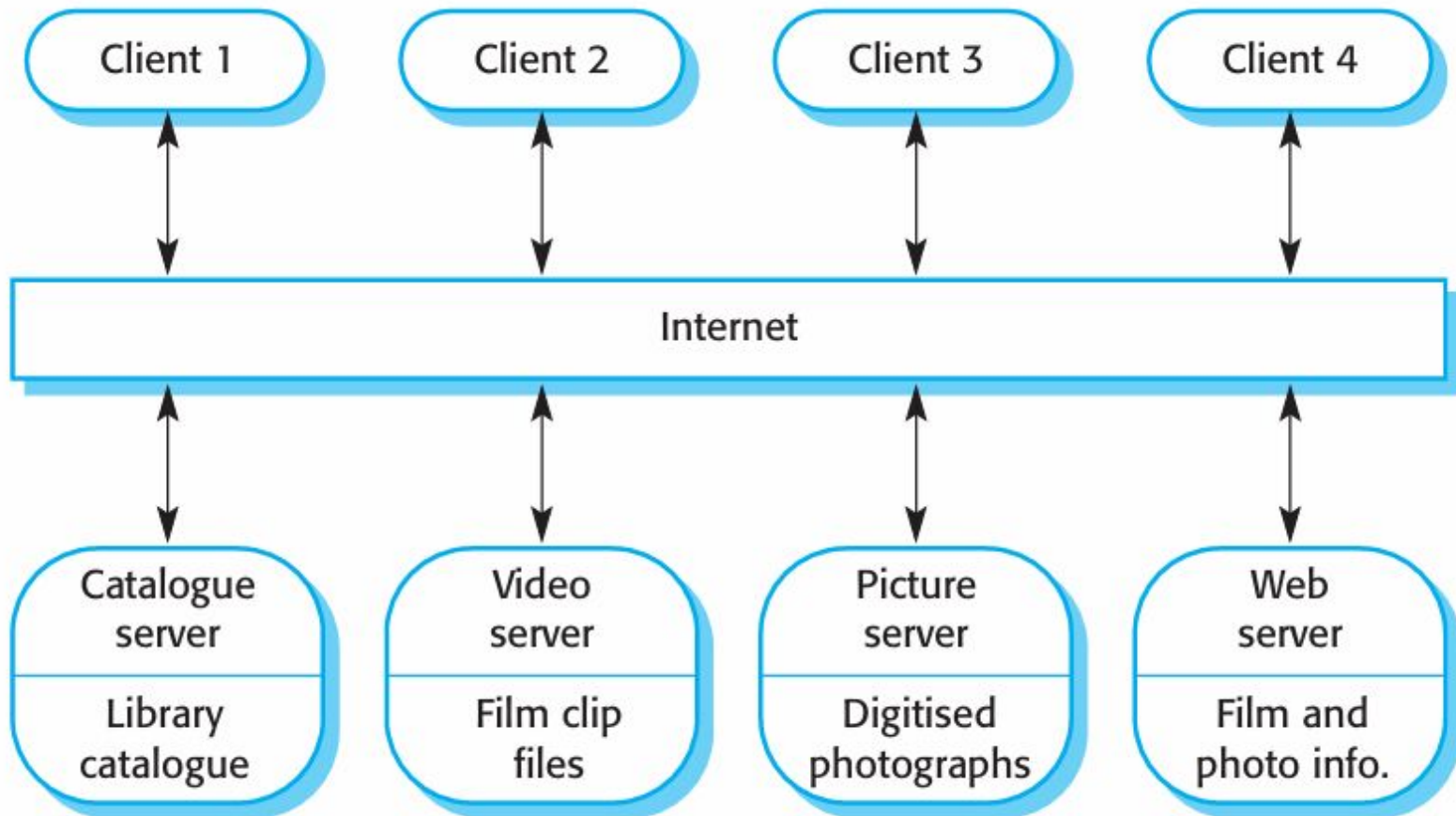
Contd..

- Activities such as backup, security, access control and recovery from error are centralized and are the responsibility of the repository manager.
However, -----
- It is straight forward to integrate new sub-systems given that they are compatible with the agreed data model.
- However, it may be difficult to distribute the repository over a number of machines.

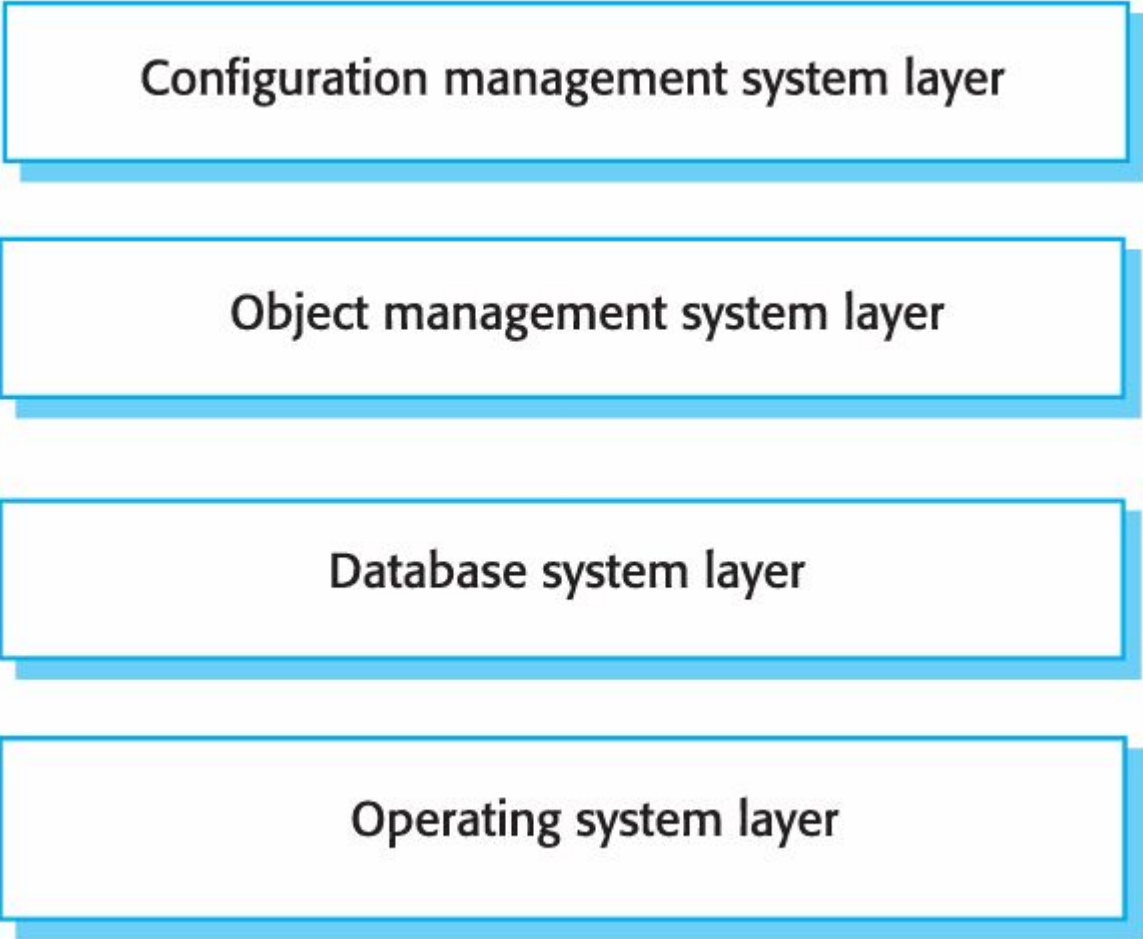
2. The Client-Server Model: The client–server architectural model is a system model where the system is organized as a set of services and associated servers and clients that access and use the services. Major components include:

- A set of servers that offer services to other sub-systems.
- A set of clients that call on the services offered by servers.
- A network that allows the clients to access these services.

Client-Server model example



3. The layered architecture: The layered model of an architecture (sometimes called an abstract machine model) organizes a system into layers.



Configuration management system layer

Object management system layer

Database system layer

Operating system layer

Figure 11.4 Layered model of a version management system

- **Advantages:** It supports the incremental development of systems.

- This architecture is also changeable and portable.

Disadvantages: A disadvantage of the layered approach is that structuring systems in this way can be difficult. Inner layers may provide basic facilities, such as file management, that are required at all levels.

- Performance can also be a problem because of the multiple levels of command interpretation that are sometimes required. If there are many layers, a service request from a top layer may have to be interpreted several times in different layers before it is processed.

Modular Decomposition Styles

Modular Decomposition Styles

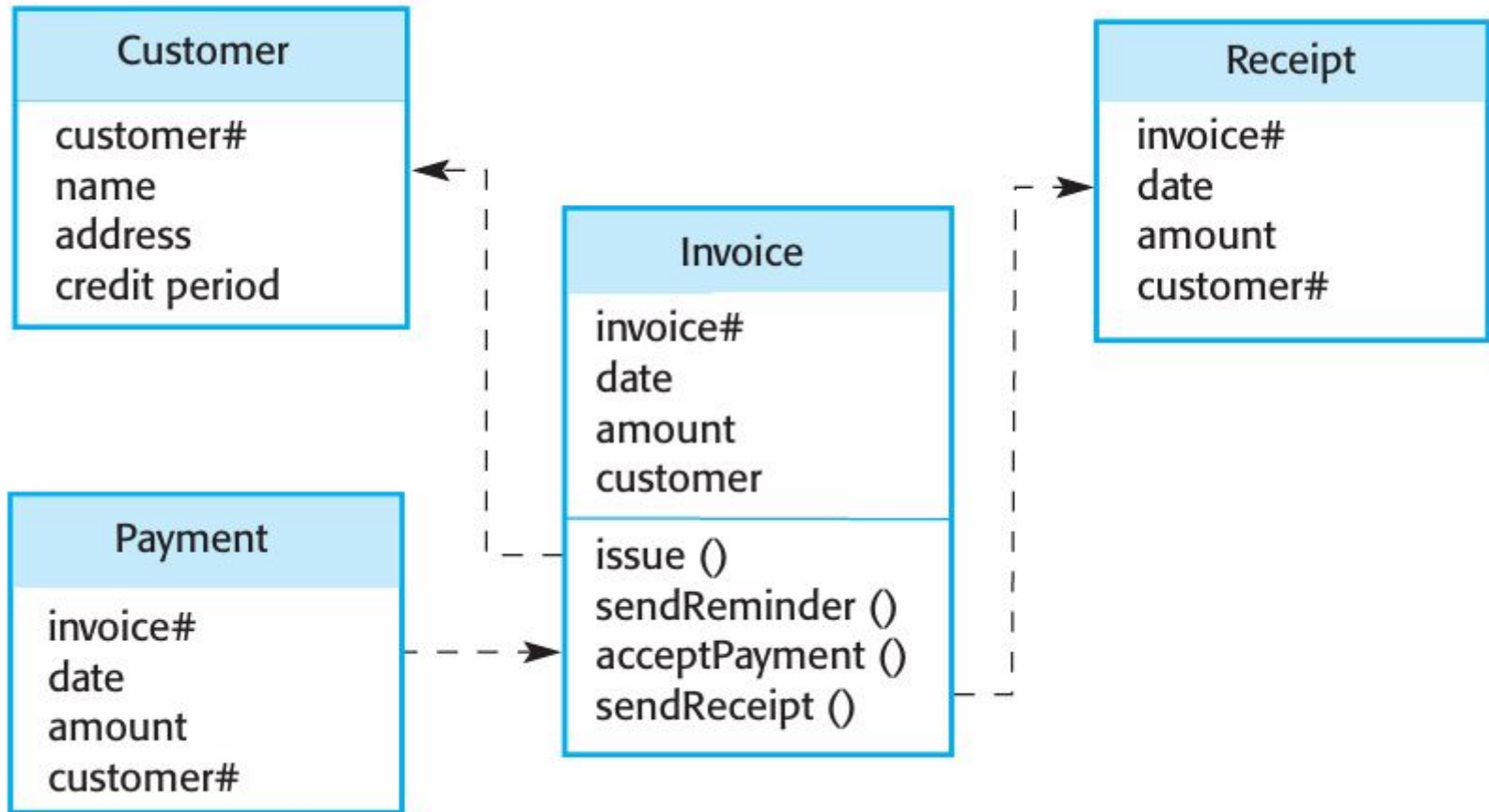
- Once all the sub-systems are identified, the next important task is to divide the sub-system into modules. Sub-system and module do not have a clear distinction, however, the following points can be noted:
 - A sub-system is a system in its own right whose operation does not depend on the services provided by other sub-systems.
 - A module is normally a system component that provides one or more services to other modules. It makes use of services provided by other modules. It is not normally considered to be an independent system.

Methods for Decomposition

- 1. Object-Oriented decomposition:** The sub-system is decomposed into a set of objects that communicate with each other.
- 2. Function-Oriented Pipelining:** The system is decomposed into functional modules that accept data and produce output.

1. Object-Oriented Decomposition

- Object oriented model structures the system into a set of loosely coupled objects with well-defined interfaces.
- Objects offer services to other objects through their interfaces.



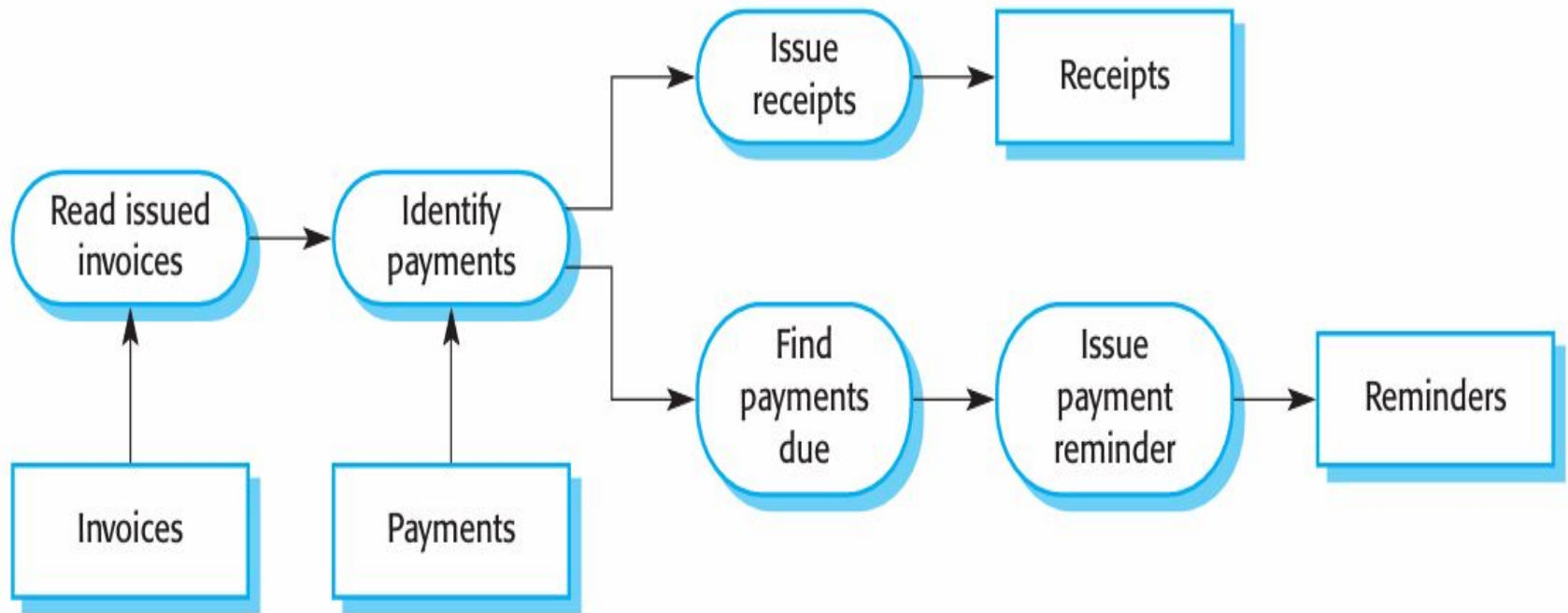
Object-Oriented model for invoice processing system

Advantages and Disadvantages

- Objects are loosely coupled, the implementation of objects **can be modified without affecting other objects.**
- Objects are often representations of real-world entities so the structure of the system is readily understandable.
- Objects are reusable.
- However, there are some disadvantages
 - Change in the interface
 - More complex entities are difficult to represent as single objects.

2. Function Oriented Pipelining

- In a function oriented pipeline, the input data undergoes functional transformations to produce the output.
- The data flows from one function to another and gets transformed.
- The name pipeline comes from Unix system's pipe.



Function –oriented pipeline for Invoice Processing System

- **Advantages:**

1. It supports the reuse of transformations.
2. It is intuitive in that many people think of their work in terms of input and out put processing.
3. Evolving the system by adding new transformations is usually straightforward.
4. It is simple to implement either as a concurrent or a sequential system.

- **Disadvantages:**

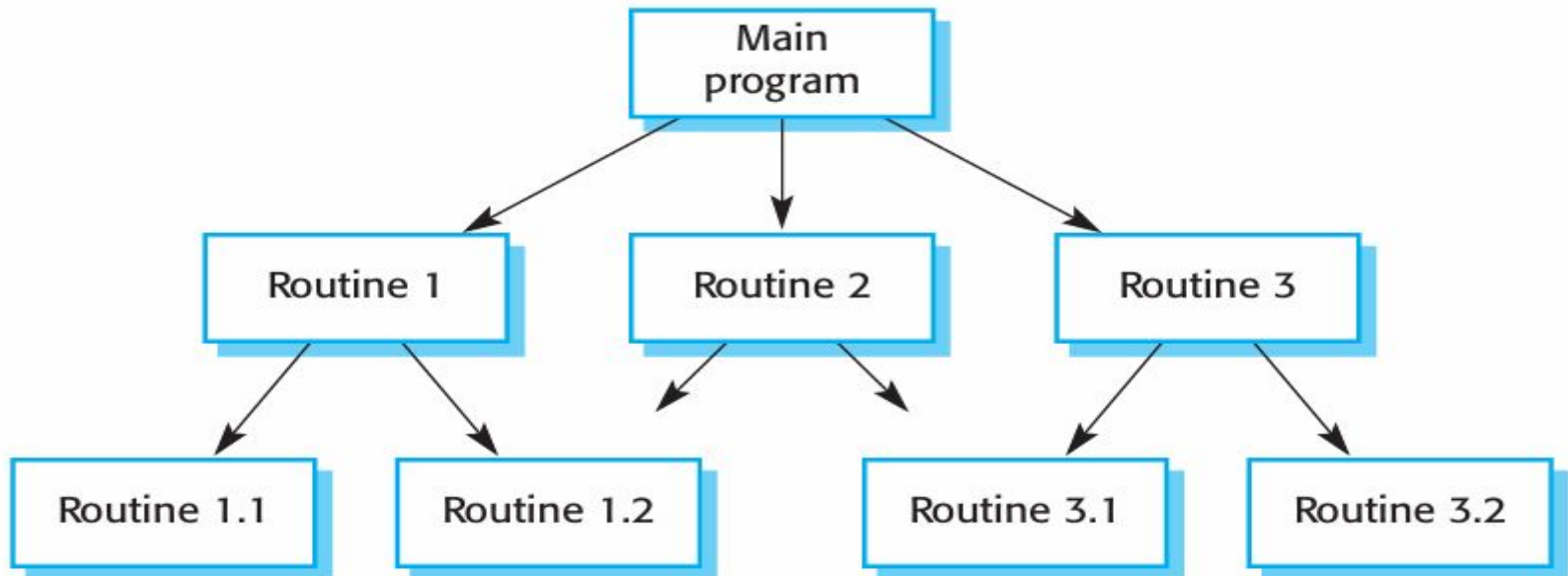
1. The principal problem with this style is that there has to be a common format for data transfer.
2. Interactive systems are difficult to write using the pipelining model because of the need for a stream of data to be processed.

Control Styles

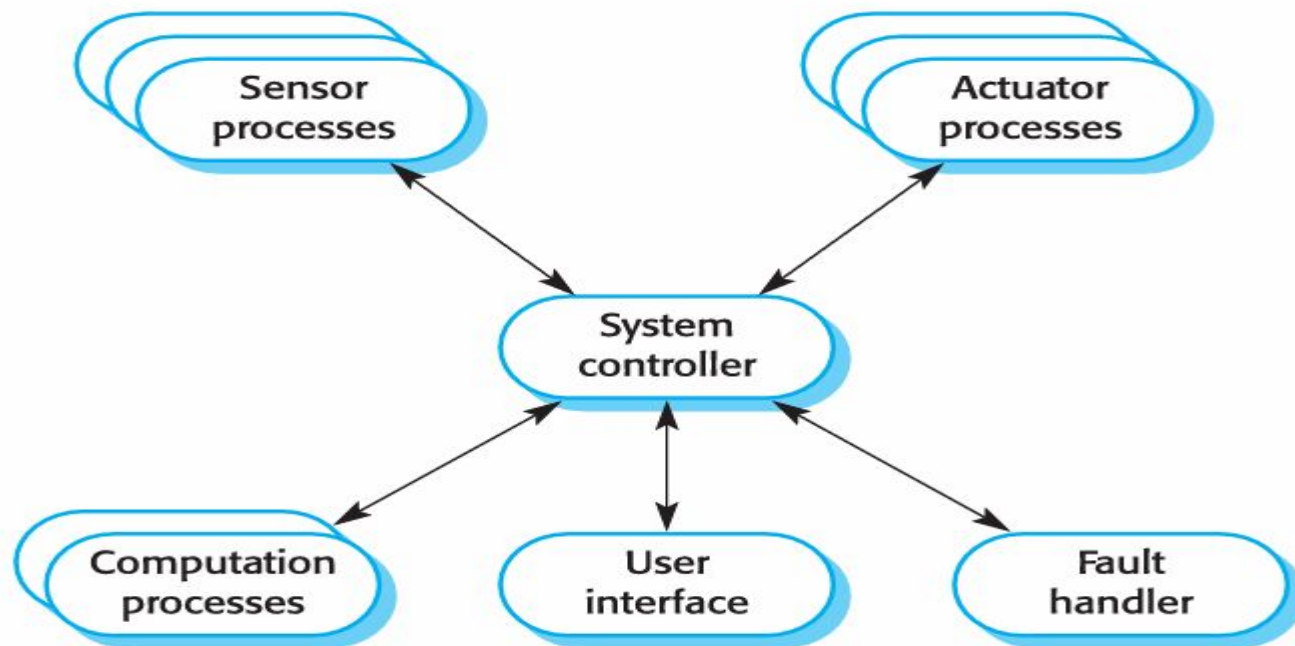
- Sub-systems must be controlled so that their services are delivered to the right place at the right time.
- There are two generic control-styles used:
 - **Centralized Control:** One sub-system has overall responsibility for control and starts and stops other sub-systems.
 - **Event-based control:** Rather than control information being embedded in a sub system, each sub-system can respond to externally generated events.

1. Centralized Control

- Centralized control models are of two types:
 - The Call-Return Model:** This is the top-down sub-routine model. The control starts at the top of a subroutine hierarchy and, through subroutine calls, passes to lower levels in the tree.



b. The Manager Model: This is applicable to concurrent systems. One system component is designated as a system manager and controls the starting, stopping and coordination of other system processes.



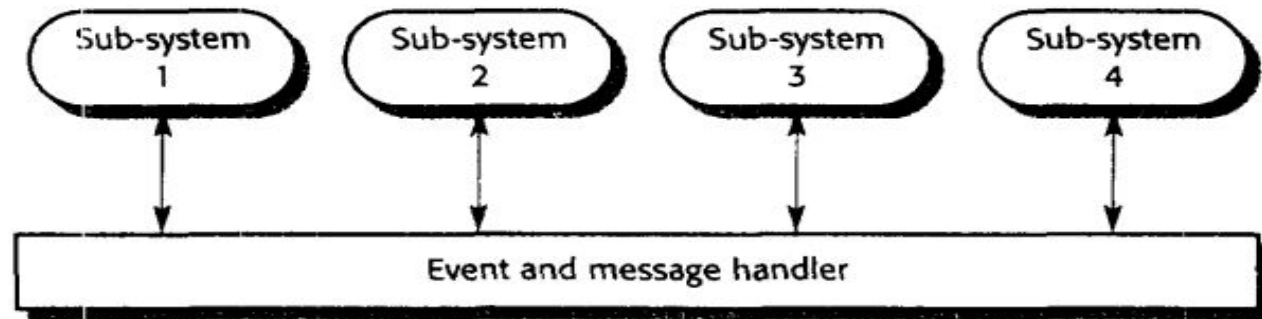
2. Event Driven Systems

- The event driven systems are controlled by **external events**.
- An event could be an internal signal or an external signal such user input. These events are asynchronous most of the times.
- Examples of event driven systems: Remote control system, text editors, AI based production systems etc.

Types of event-driven models

- **Broadcast models:** In these models, an event is broadcast to all sub-systems. Any sub-system that has been programmed to handle that event can respond to it.
- **Interrupt-driven models:** These are exclusively used in real-time systems where external interrupts are detected by an interrupt handler. They are then passed to some other component for processing.

1. Broadcast Models



- The above model is based on the selective broadcast model.
- Every sub-system registers their interests in specific events, and respond only when those events occur.

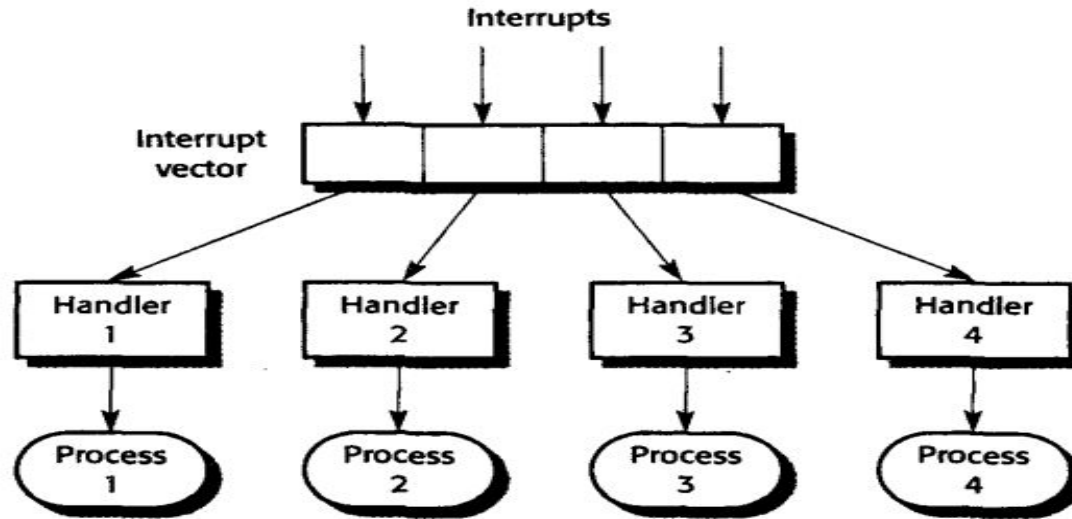
Contd..

- The event and message handler maintains an event register that has a list of sub-systems and their interested event.
- When an event occurs, the events handlers consults the event register and redirects the event to the concerned sub-system.
- Example: **Multiplayer Online Games**
- **Scenario:** When a player moves or attacks, an event is broadcast to all other players.
- **Response:** Nearby players and NPCs update their behavior or visuals.
- **Why It Fits:** Keeps the game world synchronized among clients.

Advantages and disadvantages

- Advantage: It is easy add new sub-systems. The sub-system must only have to register itself with its interest.
- Disadvantage: Sub-systems don't know when events have to be handled. If they are executing some other task, then there might be delay in handling.
 - Also, if multiple sub-systems have registered for the same event, there will be clashes.

2. Interrupt driven model



- An interrupt is high priority event that is implemented in the hardware.
- There are a known number of interrupt types with a handler defined for each type. Each type of interrupt is associated with the memory location where its handler's address is stored.

Contd..

- When an interrupt of a particular type is received, a hardware switch causes control to be transferred immediately to its handler. This interrupt handler may then start or stop other processes in response to the event signaled by the interrupt.
- This model is mostly used in real-time systems where an immediate response to some event is necessary

Advantages and disadvantages

- The advantage of this approach is that it allows very fast responses to events to be implemented.
- Its disadvantages are that it is complex to program and difficult to validate. It may be impossible to replicate patterns of interrupt timing during system testing. It can be difficult to change systems developed using this model.

- **Example**
- **Medical Monitoring Devices**
- **Scenario:** Heart rate monitor detects irregular heartbeat.
- **Interrupt:** Immediately triggers an alarm or sends data to medical staff.
- **Why It Fits:** Needs *immediate*, prioritized response without polling.

Object oriented design(Part- 2)

Part 2: Object Oriented Design

Object-Oriented Design is the process of planning a software system by modeling it as a collection of **interacting objects**, each defined by their **class**, **attributes**, and **methods**.

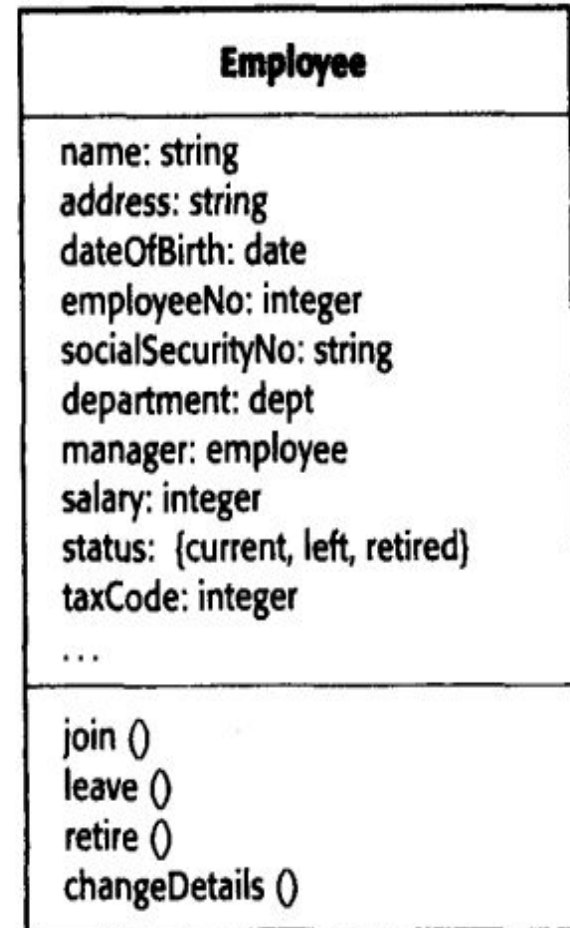
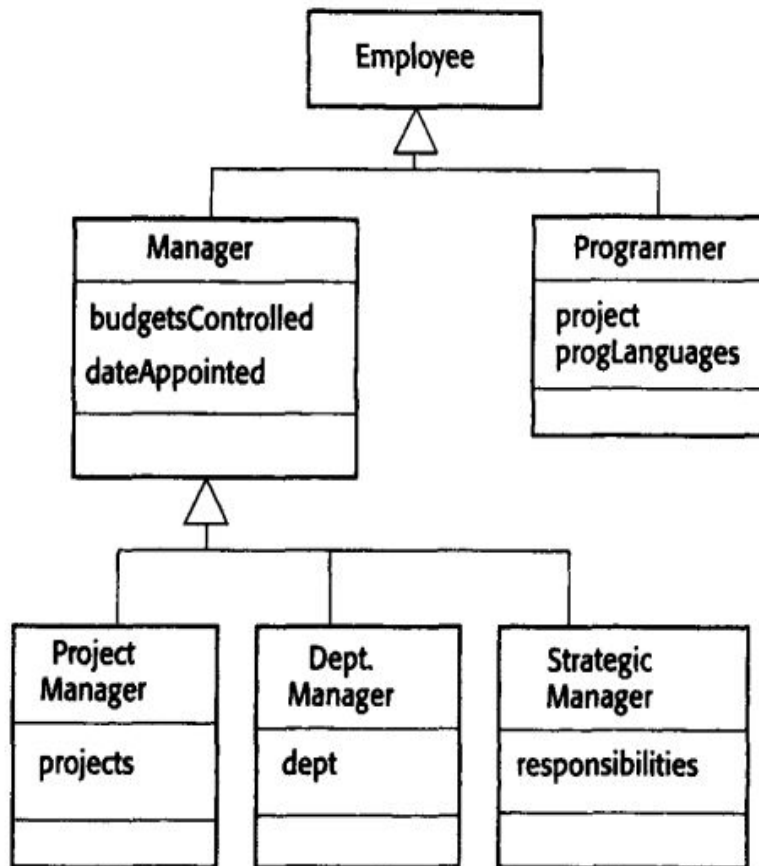
Key Principles of Object-Oriented Design:

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

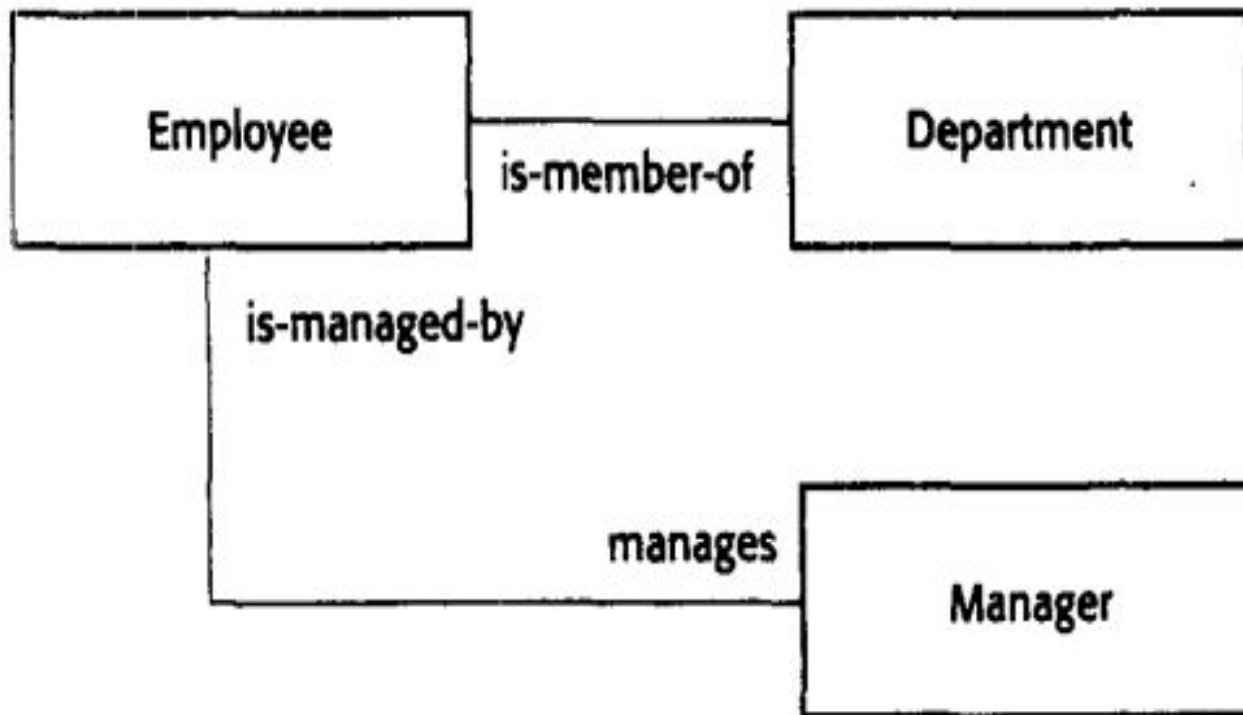
Objects and Object Classes

- *An object is an entity that has a state and a defined set of operations that operate on that state. The state is represented as a set of object attributes. The operations associated with the object provide services to other objects (clients) that request these services when some computation is required.*
- *Objects are created according to an object class definition. An object class definition is both a type specification and a template for creating objects. It includes declarations of all the attributes and operations that should be associated with an object of that class.*

UML notation



Association



Concurrent Objects

- Objects that can execute in parallel are called concurrent objects.
- When an object requests a service from another object, other objects may execute in parallel as they are independent objects.
- There are two types of concurrent object implementations:
 - Servers
 - Active Objects

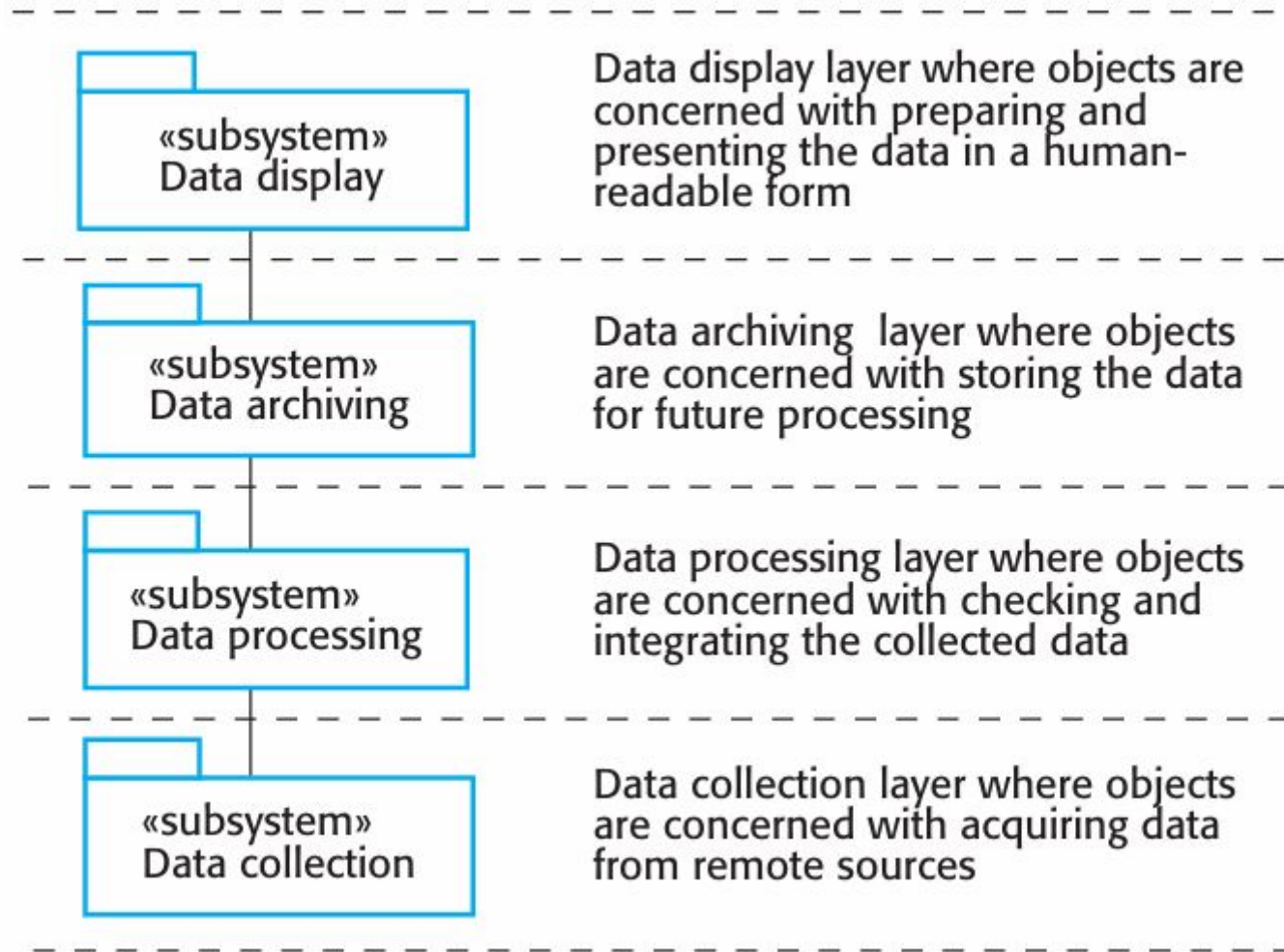
Object Oriented Design Process

- Phases in OO design process:
 - 1. Understand and define the context and the modes of use of the system.
 - 2. Design the system architecture.
 - 3. Identify the principal objects in the system.
 - 4. Develop design models.
 - 5. Specify object interfaces

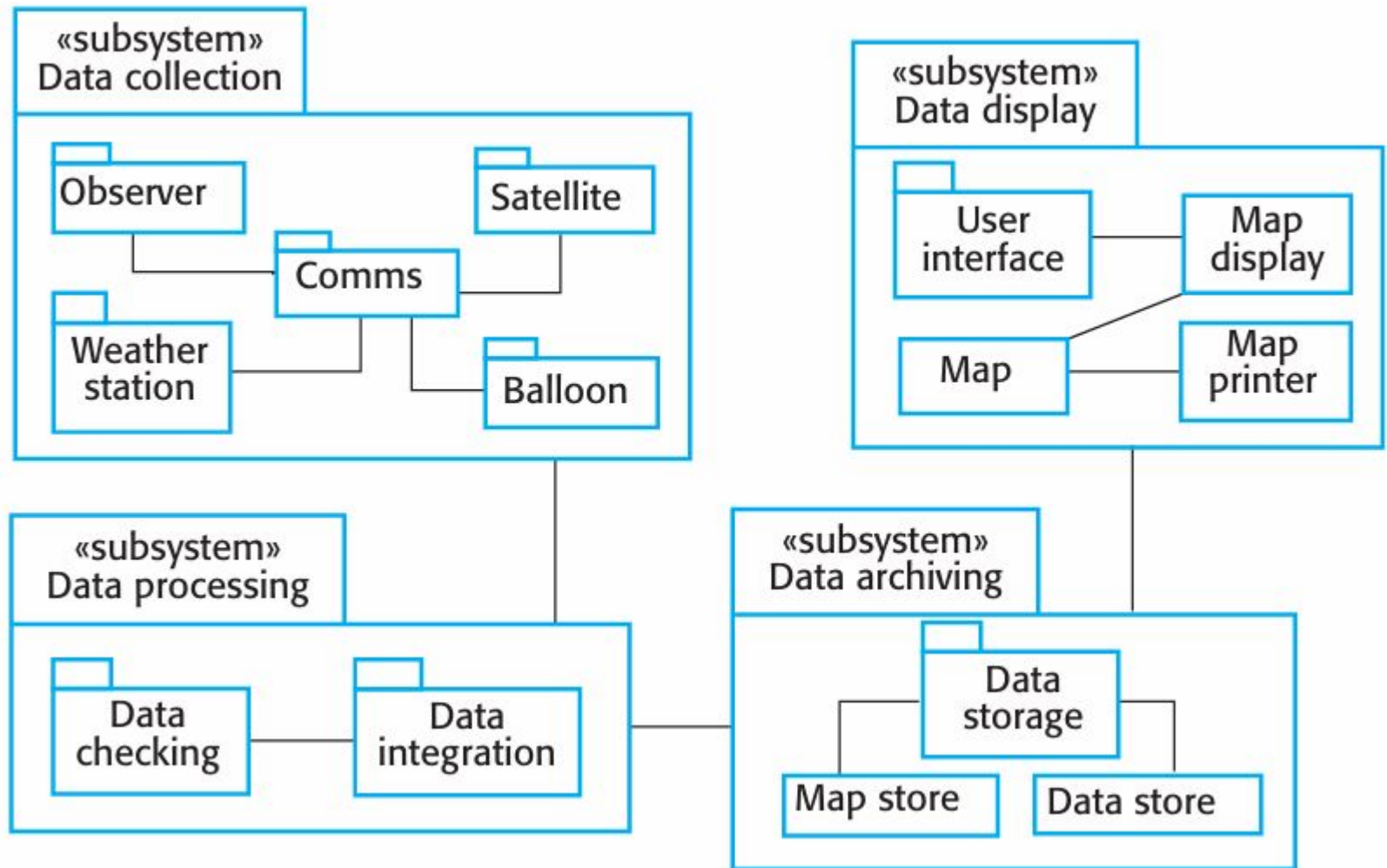
Ex: Weather Mapping System

- A weather mapping system is required to generate weather maps on a regular basis using data collected from remote, unattended weather stations and other data sources such as weather observers, balloons and satellites. Weather stations transmit their data to the area computer in response to a request from that machine.
- The area computer system validates the collected data and integrates the data from different sources. The integrated data is archived and, using data from this archive and a digitized map database, a set of local weather maps is created. Maps may be printed for distribution on a special-purpose map printer or may be displayed in a number of different formats.

Layered architecture for weather mapping system



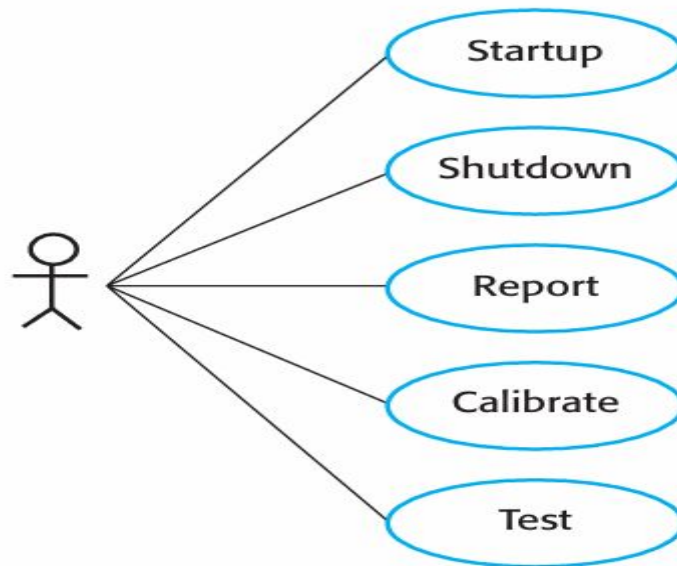
UML Package diagram of the sub-systems



1. System Context and use cases

From this point, we will consider the weather station sub-system, which is a part of the data collection sub-system.

Use cases for weather station:



Contd..

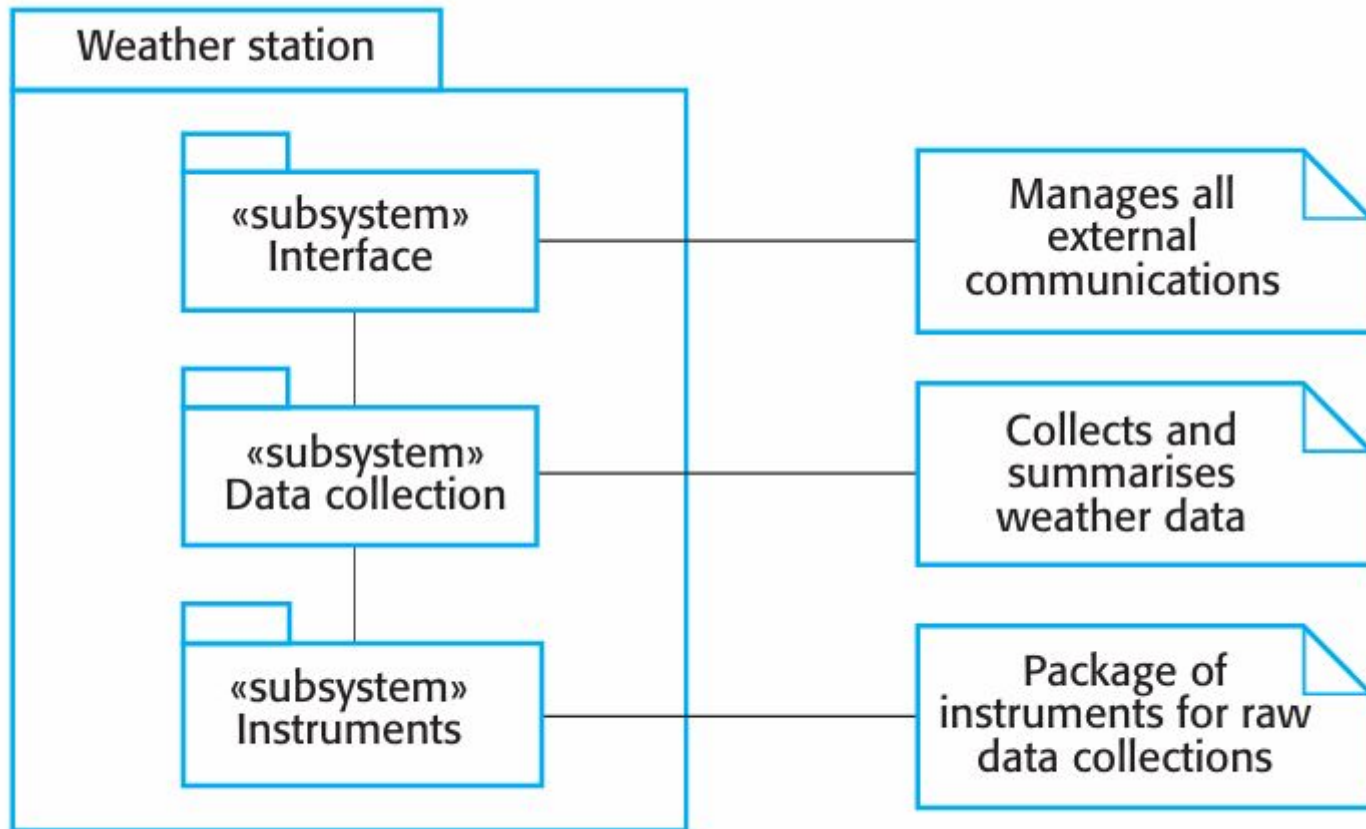
- Each of the use cases can be considered as a scenario and we can describe it using structured natural language.

System	Weather station
Use-case	Report
Actors	Weather data collection system, Weather station
Data	The weather station sends a summary of the weather data that has been collected from the instruments in the collection period to the weather data collection system. The data sent are the maximum, minimum and average ground and air temperatures, the maximum, minimum and average air pressures, the maximum, minimum and average wind speeds, the total rainfall, and the wind direction as sampled at five-minute intervals.
Stimulus	The weather data collection system establishes a modem link with the weather station and requests transmission of the data.
Response	The summarised data is sent to the weather data collection system.
Comments	Weather stations are usually asked to report once per hour but this frequency may differ from one station to another and may be modified in future.

Contd..

- The use case description helps us to identify the objects, their attributes and operations within that use case.

2. Architectural Design



Object Identification

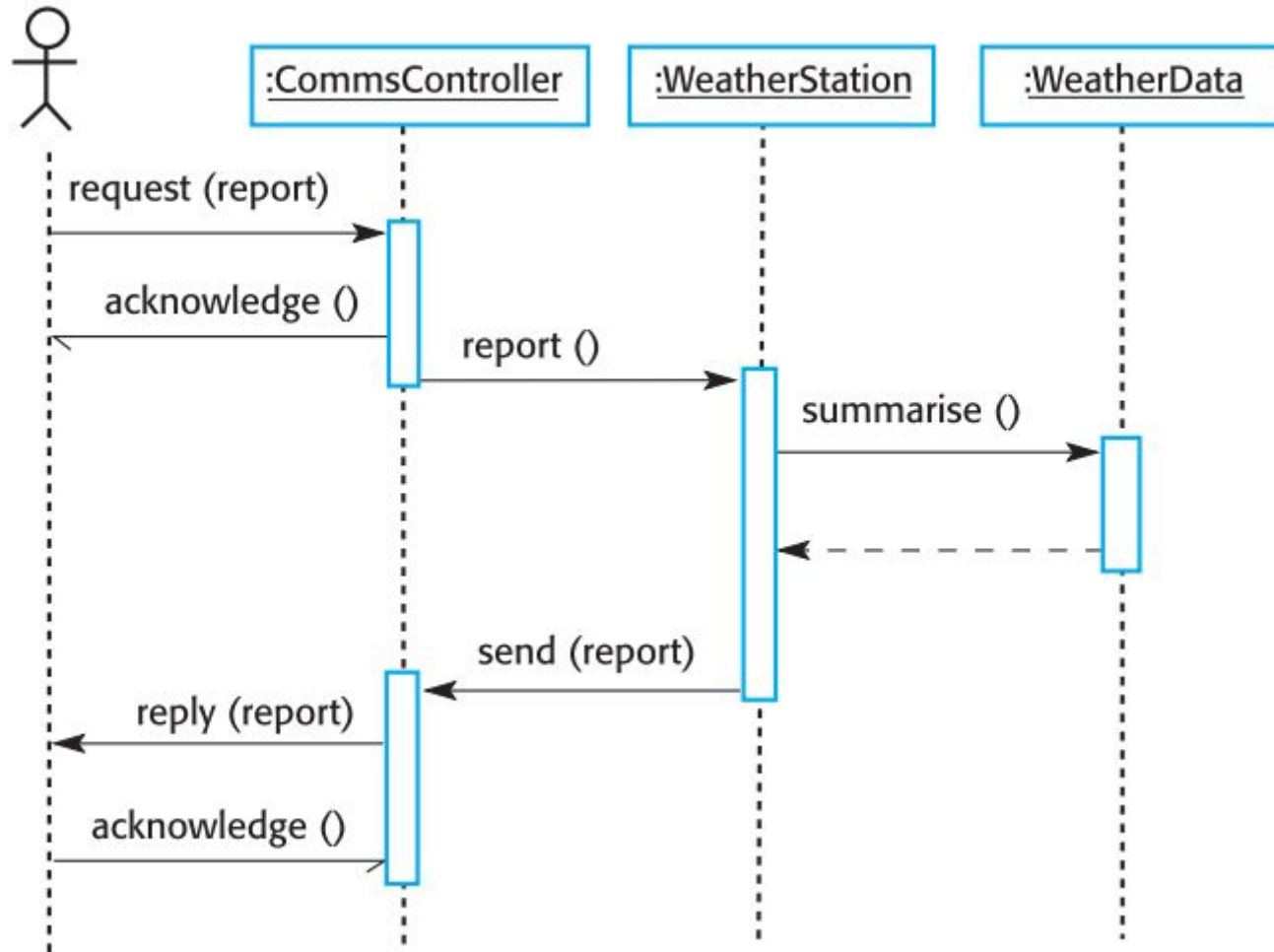
OO Design models

- Design models show the objects or object classes in a system and, where appropriate, the relationships between these entities.
- An important step in the design process, therefore, is to decide which design models that you need and the level of detail of these models.
- There are two types of OO design models:
 - Static Models
 - Dynamic Models

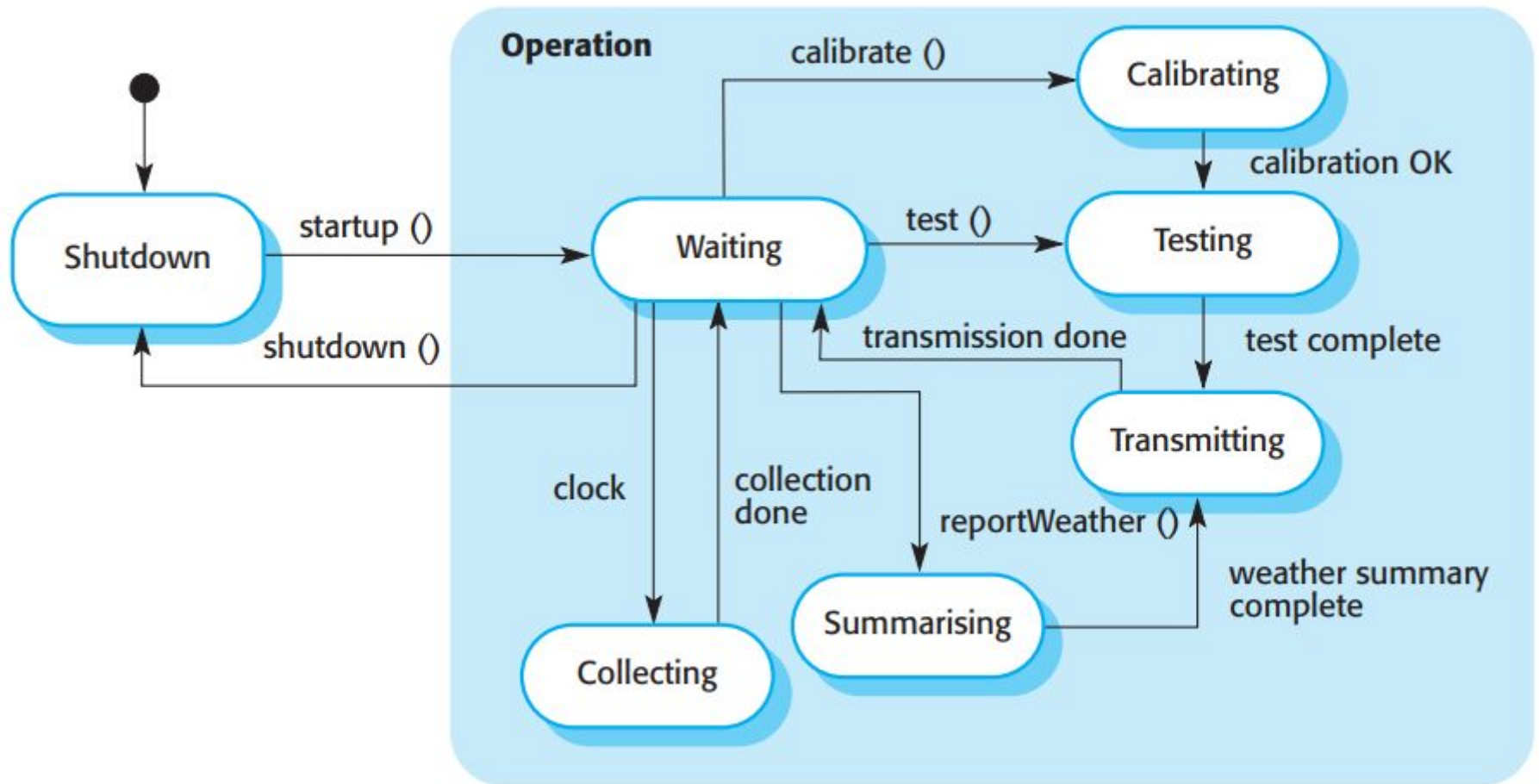
Contd..

1. **Subsystem models** that show logical groupings of objects into coherent sub-systems. These are represented using a form of class diagram where each sub-system is shown as a package. Subsystem models are static models.
2. **Sequence models** that show the sequence of object interactions. These are represented using a UML sequence or a collaboration diagram. Sequence models are dynamic models.
3. **State machine models** that show how individual objects change their state in response to events. These are represented in the UML using state chart diagrams. State machine models are dynamic models

Sequence Model for Data Collection



State Chart for a weather station



4. Object Interface Specification

```
interface WeatherStation {  
  
    public void WeatherStation () ;  
  
    public void startup () ;  
    public void startup (Instrument i) ;  
  
    public void shutdown () ;  
    public void shutdown (Instrument i) ;  
  
    public void reportWeather ( ) ;  
  
    public void test () ;  
    public void test ( Instrument i ) ;  
  
    public void calibrate ( Instrument i) ;  
  
    public int getID () ;  
  
} //WeatherStation
```